

Heuristic Monte Carlo Tree Search-Based Task Allocation for Multiple UAVs in Reconnaissance and Rescue Missions

Yueting Yang, Yi Sun, Leiming He, Xiaoting Zhou, Sibao Hu

Chinese Aeronautical Establishment, Beijing 100012, China

E-mail: ytyangcae@163.com, suny.129@qq.com, hlmwps@sina.com, 13001031322@163.com, husibo2018@163.com

Abstract—With the widespread adoption of UAV swarms, task allocation for multiple UAVs has emerged as a critical area of research. Traditional heuristic approaches often require extensive domain knowledge and are highly sensitive to parameter settings. To address these challenges, this paper introduces a Monte Carlo tree search-based Task Allocation (MCTS-TA) method to enhance UAV swarm cooperation in reconnaissance and rescue missions. The task allocation problem is reformulated as a sequential decision-making process, featuring an innovative design of the state and action spaces. Additionally, a heuristic rollout policy and an adaptive rollout number selection mechanism are proposed to improve computational efficiency. Numerical simulations demonstrate that MCTS-TA achieves superior performance compared to genetic algorithms and conventional heuristic methods.

Index Terms—Multiple UAVs, Task Allocation, Monte Carlo Tree Search

I. INTRODUCTION

With the advancement of unmanned intelligent technologies, unmanned swarms have demonstrated significant application potential in both military and civilian domains [1]. Among these, unmanned aerial vehicle (UAV) swarms stand out due to their low cost and operational flexibility, which are increasingly employed in reconnaissance and surveillance operations, including power infrastructure inspections, road monitoring, and search-and-rescue missions. Although UAV swarms significantly outperform individual drones, maximizing their benefits within limited resources presents a new challenge. As a result, multiple UAV task allocation has become a pivotal research focus, aiming to optimize the task selection and execution order involving multiple drones and diverse targets [2].

The task allocation problem for multiple drones is typically formulated as a classic combinatorial optimization problem, such as the vehicle routing problem (VRP) [3], which is proven to be NP-hard. There are numerous approaches proposed to solve task allocation problem for UAV swarms, which can be broadly classified into exact algorithms and heuristic algorithms, each with unique strengths and limitations. Exact algorithms rely on mathematical optimization to obtain precise solutions. Representative techniques include integer linear programming [4] and dynamic programming

[5]. While these methods ensure optimality, they are computationally expensive and often unsuitable for large-scale problems or those involving complex constraints. Heuristic algorithms, on the other hand, exploit domain knowledge and randomness to accelerate the solution process. Prominent methods in this category include meta-heuristic techniques (e.g., genetic algorithms [6], particle swarm optimization [7]), auction-based algorithms [8], and learning-based approaches [9]. Although heuristic algorithms generally cannot guarantee globally optimal solutions, they are capable of producing high-quality solutions efficiently, making them particularly well-suited for large-scale and complex problems. Consequently, these algorithms are widely applied in practical scenarios.

The aforementioned heuristic approaches often require carefully fine-tuned parameters (e.g., mutation rates in genetic algorithms or swarm sizes in particle swarm optimization) and substantial problem-specific knowledge to achieve satisfactory performance. In contrast, Monte Carlo tree search (MCTS) is a heuristic search algorithm extensively used in decision-making problems under uncertainty [10]. MCTS stands out for its minimal reliance on problem-specific knowledge and parameter tuning, making it more versatile and easier to adapt to a broad range of applications. MCTS was originally used in board games which is famous for AlphaGo developed by Google DeepMind [11]. Several studies have extended the application of MCTS to combinatorial optimization problems. For instance, Mandziuk *et al.* proposed an extension of MCTS to solve the capacitated vehicle routing problem (CVRP) [12]. This approach adapts the upper confidence bounds applied to trees (UCT) with a two-phase node selection mechanism. Promising results are achieved by increasing the exploration factor and selecting a well-designed internal evaluation function to balance exploration and exploitation effectively. Nguyen *et al.* leveraged MCTS to optimize package delivery systems using drones, where random sampling in the decision space leads to high-quality decisions [13]. Ma *et al.* addresses UAV-specific constraints, such as load capacity and battery energy, by integrating K-means clustering with MCTS to minimize energy consumption [14].

Although MCTS has been applied to task allocation in UAV swarms, there remains significant potential for exploration in

This project was supported by Aeronautical Science Foundation of China(Grant Number: 2024M004004001).

effectively mapping the VRP onto a game tree structure. In this paper, we propose a novel MCTS-based task allocation method for reconnaissance and rescue missions. This approach further investigates effective designs for the state-action space, rollout policy, and terminal criteria, aiming to achieve faster and more optimal solutions. Additionally, extensive numerical simulations demonstrate that our proposed method significantly outperforms both genetic algorithms (GA) and pure heuristic approaches.

The remainder of the paper is organized as follows. Section II presents a formal definition of the task allocation problem. Section III details the proposed MCTS-based task allocation method. Section IV presents the results of numerical simulations. Finally, Section V summarizes the conclusions drawn from this study.

II. PROBLEM DEFINITION

With the advancement of drone technology, drone swarms have demonstrated significant potential in reconnaissance and rescue missions. For instance, when natural disaster such as earthquakes and floods occurs, drone swarms can rapidly conduct comprehensive searches of affected areas to locate individuals awaiting rescue. Simultaneously, drone can carry and deliver essential rescue supplies to critical locations. Given this context, this paper investigates the task allocation problem for drone swarms in reconnaissance and rescue missions, aiming to enhance the operational efficiency of drone swarms. To align the study with practical applications, the task allocation problem is formulated with the following assumptions: (1) The drones depart from and return to the same airport; (2) Each drone can perform multiple tasks, but each task is assigned to only one drone; (3) The drone swarm is heterogeneous with variations in endurance, flight speed, reconnaissance capabilities, and rescue capabilities.

Further, the objective and constraint of the proposed task allocation problem are described by mathematical symbols. The set of drones is denoted as $\mathcal{U} = \{U_i\}$, $i = 1, \dots, N$. The capability and attributes of each drone are different. The flight duration, flight speed, search capability, and rescue capability are expressed as D_i^d , V_i^d , S_i^d , R_i^d , respectively. The set of tasks is denoted as $\mathcal{T} = \{T_j\}$, $j = 0, 1, \dots, M$, where the T_0 represents the departure or arrival airport. The connectivity and distance between any two tasks are expressed by a symmetric matrix $D \in \mathbb{R}^{(M+1) \times (M+1)}$, where $D_{i,j}$ represents the distance between the task T_i and T_j . Note that $D_{i,j} = 0$ means that the route from T_i to T_j is forbidden. Besides, the execution time, reward, search demand, and rescue demand of task T_j are denoted as E_j^t , W_j^t , S_j^d , and R_j^d . The allocated task sequence of U_i can be defined as $\gamma_i = (\tau_0, \tau_1, \tau_2, \dots, \tau_M)$, where τ_* is a symbol that represents the index of task. In this problem, the τ_0 and τ_M must be equal to 0, which means that the drone must start from airport and finally return to airport. Thus, the solution of this problem is a set of task sequences allocated to every drone, which can be described as $\gamma = \{\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_N\}$. Notably, γ_0 represents the set of all unassigned tasks.

The objective of this problem is to seek for a solution that minimize the maximum task completion time of all vehicles and the total reward of all unassigned tasks. The mathematical expression can be written as follow:

$$J(\gamma) = \min \left\{ \max_{\gamma_i \in \gamma} \left(\sum_{k=0}^M E_{\tau_k}^t + \frac{D_{\tau_k, \tau_{k+1}}}{V_i^d} \right) + \sum_{\tau_k \in \gamma_0} W_{\tau_k}^t \right\} \quad (1)$$

Considering practical applications, the solution must also satisfy two constraints. The first is the endurance constraint, which ensures that each drone has sufficient energy to return to the airport. It indicates that the flight time and task execution time of each drone must be less than its maximum flight endurance, which is

$$\sum_{k=0, \tau_k \in \gamma_i}^M E_{\tau_k}^t + \frac{D_{\tau_k, \tau_{k+1}}}{V_i^d} \leq D_i^d, \quad \forall i = 1, \dots, N \quad (2)$$

The second is the capability constraint, which means that only suitable drone can be dispatched to carry out the specific task. It indicates that the search and rescue demand of task must not exceed the capability of drone, which is

$$S_{\tau_k}^t \leq S_i^d, \quad \forall \tau_k \in \gamma_i, \quad \forall i = 1, \dots, N \quad (3)$$

$$R_{\tau_k}^t \leq R_i^d, \quad \forall \tau_k \in \gamma_i, \quad \forall i = 1, \dots, N \quad (4)$$

Overall, the problem setting is thoroughly introduced, which is similar with a vehicle routing problem but with more complexity. Traditional VRP algorithm can not be directly applied in this problem. Thus, a novel Monte-Carlo-Tree-Search-Based task allocation method for drone swarms is further proposed.

III. MONTE-CARLO-TREE-SEARCH-BASED TASK ALLOCATION METHOD

A. Introduction of Monte Carlo tree search

Following the remarkable success of AlphaGo, Monte Carlo Tree Search (MCTS) has garnered significant attention within the AI community [11]. However, there have been relatively few attempts to apply MCTS to combinatorial optimization problems, particularly in the context of task allocation. Actually, task allocation can be naturally transformed into a sequential decision-making problem, where each decision or selection corresponds to a node in a decision tree. MCTS is particularly well-suited for addressing this problem due to its high efficiency, its ability to escape local minima through randomization, and its capacity for parallelization.

MCTS is a heuristic search method with two major characters. First, it utilizes the lots of Monte Carlo simulation to collect outcomes for evaluate the value of state. Second, the expansion of decision tree is guided by the upper confidence bounds (UCB) strategy, which can balances exploration and exploitation factors to find the most promising directions of tree search. Based on the above basic MCTS algorithm, a series of improvement and adjustment are maded to adapt it to the proposed task allocation problem.

B. MCTS-Based UAV Task Allocation Algorithm

In this subsection, a novel MCTS-Based Task Allocation (MCTS-TA) algorithm is proposed to solve the above problem with high-efficiency and high-equality. The proposed method is based on the work of Nguyen [13], and make some improvements on state and action selection, heuristic rollout policy, and terminal state evaluation. Algorithm 1 describes the kernel idea of the MCTS-TA.

1) *State and Action Definition*: The solution process of sequential decision-making problem can be regarded as constructing a decision tree, expanding child nodes incrementally, and seeking for the optimal node. Each node on decision tree represents a state, while each edge denotes an action. In sequential decision-making problems, the definitions of state and action are crucially related to the efficiency and quality of the solution process. Generally, if the action set is overly simple, the expansion and search of the decision tree will be efficient because the expandable nodes are less. Consequently, high-quality solutions may not be obtained. For instance, in Mandziuk's work [12], the action involves selecting the i -th vehicle and then assigning the task closest to the vehicle to it. This action definition employs strong heuristic information to limit the number of executable actions to the number of vehicles. However, this definition significantly constrains the quality of the final solution. Conversely, if the action set is overly complex, it will result in an excessive number of expandable nodes, thereby expanding the decision tree excessively and reducing the solution efficiency. For example, an action is designed to select the i -th vehicle to perform the j -th task and position it at the k -th position in the task sequence. Such a definition generates an enormous set of alternative actions without employing any heuristic information, leading to very low search efficiency.

To better balance efficiency and quality, the action definition proposed in this paper involves selecting the j -th task and assigning it to the i -th drone. Simultaneously, the j -th task is automatically inserted into the optimal position within the current task sequence of the i -th vehicle. Notably, if a task is assigned to the 0-th vehicle, it indicates that the task is abandoned. For specific heuristic task insertion algorithms, please refer to ActionSeqToSolution in Algorithm 1. Furthermore, this paper defines the state as the current task allocation result, which corresponds to the task sequence of each drone. Therefore, the initial state is represented by the initial task sequence of each drone as (T_0, T_0) .

2) *Heuristic Rollout Policy*: The rollout step in the MCTS algorithm involves executing a predesign decision policy from one node to the terminal state. The final rollout result is used to evaluate the value of a node and guide the direction of the tree expansion. Notably, the accuracy of the estimated value is highly relative with the predesign policy. If the optimal policy is applied, the rollout result can reflect the true value of the node. Conversely, if a completely random strategy is adopted, the result of each rollout is merely a random sample of value distribution. Consequently, a completely random strategy

necessitates a large number of rollouts to accurately evaluate the value of node. Based on these considerations, this paper proposes a heuristic rollout policy, which comprises two parts: heuristic sampling of candidate tasks and heuristic sampling of candidate vehicles.

The specific method for heuristic sampling of candidate tasks involves calculating the score of each candidate task, converting the task scores into a probability distribution using the *softmax* function, and then randomly sampling the task index based on this distribution. The score of a candidate task is primarily determined by the ratio of the task's reward to its time. A higher ratio results in a higher score and a greater probability of being selected. The task score for T_j is calculated as follows:

$$s_j^t = \frac{W_j^t}{E_j^t + (D_{0,j} + D_{j,0})/V_{avg}^d}, \quad V_{avg}^d = \frac{1}{N} \sum_{i=1}^N V_i^d \quad (5)$$

The method for heuristic sampling of candidate vehicles is similar to that for candidate tasks. The primary difference lies in the calculation method for the score. Since the task has already been selected, the score of candidate vehicles is mainly determined by the minimum distance between the vehicle route and newly selected task. A smaller minimum distance indicates that the candidate vehicle is more likely to complete the task in a shorter time. The vehicle score for U_i with the selected task T_j can be calculated as follows:

$$s_i^d = \max_{\tau_k \in \gamma_i} \frac{1}{D_{\tau_k,j} + D_{j,\tau_{k+1}}} \quad (6)$$

3) *Terminal State Evaluation*: In MCTS algorithm, it is imperative to evaluate the victory or defeat of the terminal state to update the value of each node. Unlike the games such as Go, the cost of the final solution rather than a win or loss is evaluated in task allocation problems. Strictly speaking, if the solution corresponds to the optimal solution, it can be assessed as a victory; otherwise, it is considered a failure. However, this strict definition faces two major difficulties. First, the optimal solution is unknown before the problem is solved. Second, many solutions found during rollout stage cannot be effectively utilized because most of them are failures. To address these issues, the best newly discovered solution is applied as benchmark to evaluate the victory or defeat of solution during the MCTS search process. Besides, the solutions that are better than the current benchmark are deemed victories, while the solution close to the benchmark can also earn scores. Thus, the value of terminal state or a final solution is evaluated as:

$$R(\gamma) = \begin{cases} 1 & \text{for } J(\gamma) < J_{best} \\ \left(\frac{\alpha J_{best}}{J(\gamma)} - 1 \right)^2 & \text{for } J_{best} < J(\gamma) < \alpha J_{best} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where $\alpha > 1.0$ and J_{best} denotes the cost of current best solution.

IV. NUMERICAL EXPERIMENTS

A. Experiment Setting

In order to evaluate the performance of the proposed method, a series of datasets with different numbers of tasks and UAVs are generated, e.g., Task-05-Veh-02, Task-10-Veh-03, Task-20-Veh-05, and Task-50-Veh-10. Notably, the Task- X -Veh- Y dataset means that X tasks and Y drones are need to be processed. Besides, each dataset contains 10 groups of data. The location of the depot is $[50, 50]$ km and the speed of the drone is 60 km/h. The other parameters are randomly sampled from uniform distribution in a given interval. The sample interval of these parameters are listed in Table I.

TABLE I
DATASET PARAMETER SAMPLE RANGE

Parameter	Range
Coordinate of task	$x, y \in [0, 100]$ km
Reconnaissance and rescue demands of task	$[0, 0.7]$
Reward of task	$[50, 100]$
Execution time of task	$[10, 30]$ min
Reconnaissance and rescue capabilities of UAV	$[0.3, 1.0]$
Endurance time	$[200, 300]$ min

Two numerical experiments are performed in this section. The first experiment verifies the influence of algorithm hyperparameters, namely, the rollout number. The second experiment verifies the performance of the algorithm proposed in this paper compared with the traditional methods such as genetic algorithm (GA) and pure heuristic (PH) approach.

B. Performance Evaluation for MCTS Super-Parameters

In this subsection, the impact of rollout number on MCTS behavior is studied. Rollout number refers to the simulation number of times when the action reward is evaluated. Generally, a higher rollout number leads to a more precise evaluation of the action rewards, which typically results in improved outcomes. However, the computation cost is also increased with the rollout number.

To balance the performance and computation cost, the various rollout number, including 20, 50, 100, 200, and 400, are tested based on the datasets of Task-05-Veh-02, Task-10-Veh-03, and Task-20-Veh-05. Besides, a rollout number selection policy (RNSP) is proposed here, which sets the rollout number of every MCTS loop equal to four times the product of the number of remaining tasks and vehicles. Each experiment is executed in 10 times. The average value of the final cost and running time are shown in Figure 1.

As shown as the figure, the cost significantly decreases with an increase in the rollout number, while the average run time substantially increases. For the instance in Task-10-Veh-03, the cost across multiple runs becomes concentrated when the rollout number reaches 200. For the instance in Task-20-Veh-05, the cost continues to decrease when the rollout number reaches 400, but the rate of decrease becomes small. Additionally, the distribution of cost becomes more concentrated. This indicates

Algorithm 1: MCTS-based Task Allocation Method

```

1 Function MCTS_TA():
2   Initialize action sequences  $s_{mcts} \leftarrow []$ ,  $s_{best} \leftarrow []$ 
3   while there exists unassigned tasks do
4      $a, s_{final} \leftarrow \text{MCTSActionSearch}(s_{mcts})$ 
5      $s_{mcts} \leftarrow [s_{mcts}, a]$  (Add action  $a$  into  $s_{mcts}$ )
6      $\gamma_{best} \leftarrow \text{ActionSeqToSolution}(s_{best})$ 
7      $\gamma_{final} \leftarrow \text{ActionSeqToSolution}(s_{final})$ 
8     if  $J(\gamma_{final}) \leq J(\gamma_{best})$  then
9        $s_{best} \leftarrow s_{final}$ 
10    end
11  end
12   $\gamma_{mcts} \leftarrow \text{ActionSeqToSolution}(s_{mcts})$ 
13  if  $J(\gamma_{mcts}) \leq J(\gamma_{best})$  then
14     $\gamma_{best} \leftarrow \gamma_{mcts}$ 
15  end
16  return  $\gamma_{best}$ 
17 End Function
18
19 Function MCTSActionSearch( $s$ ):
20   Initialize  $R_{best} \leftarrow \infty$ ,  $s_{best} \leftarrow []$ 
21   Create node  $n_{root}$  with action sequence  $s$ 
22   while there are enough time left do
23      $n_{select} \leftarrow \text{Traverse}(n_{root})$ 
24      $n_{expand} \leftarrow \text{Expand}(n_{select})$ 
25      $R_{final}, s_{final} \leftarrow \text{Rollout}(n_{expand})$ 
26     Backpropagate( $n_{expand}, R$ )
27     if  $R_{final} \leq R_{best}$  then
28        $s_{best} \leftarrow s_{final}$ ,  $R_{best} \leftarrow R_{final}$ 
29     end
30   end
31   Choose the best child of  $n_{root}$  and the action  $a$ 
32   return  $a, s_{best}$ 
33 End Function
34
35 Function ActionSeqToSolution( $s$ ):
36   Initialize  $\gamma_0, \dots, \gamma_N \leftarrow [T_0, T_0]$ 
37   for  $a_i^j$  in  $s$  do
38     //  $a_i^j$  represents allocate  $T_j$  to  $U_i$ 
39     Choose the task list  $\gamma_i$  of  $U_i$ 
40     Initialize  $J_{best} \leftarrow \infty$ ,  $\gamma_i^{best} \leftarrow \gamma_i$ 
41     for  $k = 1$  to  $\text{size}(\gamma_i) - 1$  do
42        $\gamma_i^k \leftarrow \text{Add } T_j \text{ into } k\text{-th position of } \gamma_i$ 
43       Calculate the cost function  $J_k$  of  $\gamma_i^k$ 
44       if  $J_k \leq J_{best}$  then
45          $\gamma_i^{best} \leftarrow \gamma_i^k$ ,  $J_{best} \leftarrow J_k$ 
46       end
47     end
48      $\gamma_i \leftarrow \gamma_i^{best}$ 
49   end
50   return  $\gamma = [\gamma_0, \gamma_1, \dots, \gamma_N]$ 
51 End Function

```

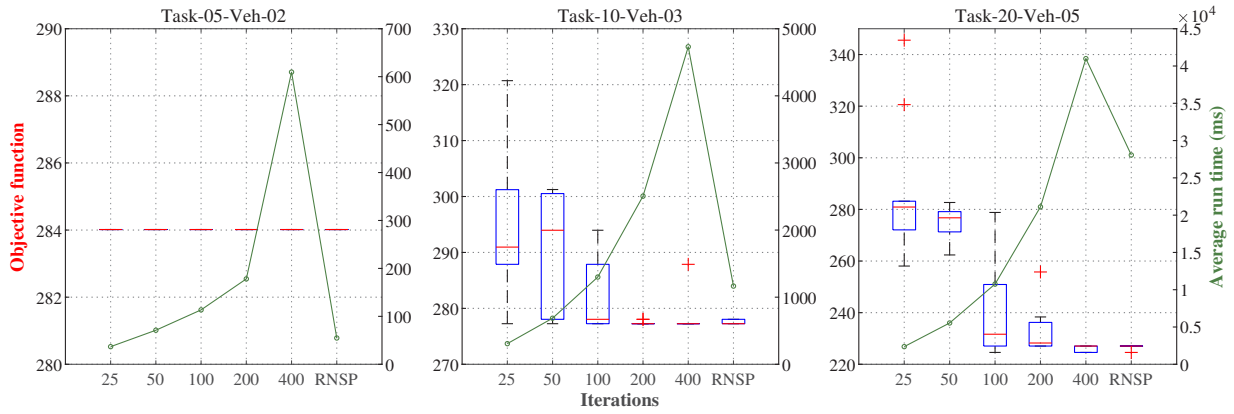



Fig. 1. Performance of MCTS-UTA on various rollout number.

that there exists a critical threshold beyond which increasing the rollout number can not result in significant improvement in solution quality. Based on this observation, the RNSP is developed. It is obviously that this policy achieves solution quality comparable to that obtained with a rollout number of 400, but with significantly reduced computational time.

C. Performance Comparison for Different Algorithm

In this subsection, the performance of proposed algorithm MCTS-TA (MT) is evaluated under multiple datasets. The proposed method is compared with MCTS method without heuristic rollout policy (MWH), genetic algorithm (GA), and pure heuristic (PH) method. More specifically, the implementation of GA refers to an open source code repository¹, which is a modified version to adapt the specific problem form in this paper. The PH method means that the proposed heuristic rollout policy is adopted directly without any random sampling. Four methods are tested on the same benchmark setting. Each problem in each datasets is run 10 times, and the minimum, maximum, and average of the final costs are recorded. The results are shown in Table II.

Overall, the proposed MT method outperforms the other three approaches in terms of minimum, maximum, and average costs. However, in the case of the minimum values, there are four instances where the MT method is slightly inferior to the MWH method. Owing to the randomness in the rollout of MCTS, it is possible that a random rollout policy could surpass heuristic methods in a single test. Specifically, in the Task-20-Veh-05 dataset, the MT method only falls short of the MWH method in terms of minimum cost but demonstrates significantly superior performance in both maximum and average cost. This result shows the superiority of the MT method in a large sample testing environment.

Furthermore, as shown in the table, the solution quality of the four methods ranks as follows: MT > MWH > GA > PH. Several conclusions can be drawn from these results. First, the heuristic rollout policy significantly enhances the performance

of the MCTS algorithm. Second, the MCTS-based algorithm outperforms the GA algorithm in the proposed problem. The possible reasons includes the use of a random initial solution generation strategy, which lacks heuristic guidance, as well as the complexity of the problem constraints, leading to low search efficiency. Third, when the number of tasks is relatively small, the performance of PH is comparable to the other three methods. However, when the problem scale increases to 20 tasks, the MT, MWH, and GA methods all significantly outperform the PH method. This indicates that it is insufficient to rely solely on heuristic strategies for complex problems. Instead, search and optimization methods must be employed.

Finally, Figure 2 visually presents the task allocation results obtained by the proposed method on the Task-50-Veh-10 dataset.

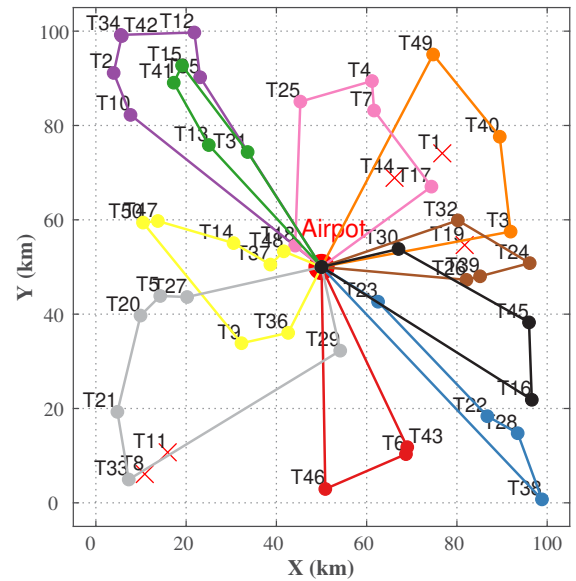


Fig. 2. Task allocation result of one instance in Task-50-Veh-10.

¹<https://github.com/IRB-Lab/py-ga-VRPTW>

TABLE II
COMPARISON OF SOLUTION COSTS ACROSS VARIOUS APPROACHES

Instance	Cost Min.				Cost Max.				Cost Avg.			
	MT	MWH	GA	PH	MT	MWH	GA	PH	MT	MWH	GA	PH
Task-05-veh-02-01	147.31	147.31	174.06	147.31	147.31	147.31	174.06	147.31	147.31	147.31	174.06	147.31
Task-05-veh-02-02	339.99	339.99	462.68	360.07	339.99	339.99	462.68	360.07	339.99	339.99	462.68	360.07
Task-05-veh-02-03	317.00	317.00	419.59	317.00	317.00	317.00	419.59	317.00	317.00	317.00	419.59	317.00
Task-05-veh-02-04	208.12	208.04	239.34	208.04	208.12	208.04	239.34	208.04	208.12	208.04	239.34	208.04
Task-05-veh-02-05	190.58	190.58	240.13	190.58	190.58	190.58	240.13	190.58	190.58	190.58	240.13	190.58
Task-10-veh-03-01	286.23	286.23	405.37	286.23	286.23	316.90	405.37	316.90	286.23	294.06	405.37	300.02
Task-10-veh-03-02	208.90	208.90	277.72	208.90	208.90	217.28	277.72	213.34	208.90	210.88	277.72	210.87
Task-10-veh-03-03	277.25	277.25	352.42	278.36	278.04	296.20	352.42	306.47	277.49	285.65	352.42	289.17
Task-10-veh-03-04	258.04	258.04	316.63	258.04	262.30	258.77	316.63	280.70	259.59	258.19	316.63	267.63
Task-10-veh-03-05	307.07	307.07	340.88	307.07	307.07	311.11	340.88	330.71	307.07	307.47	340.88	313.89
Task-20-veh-05-01	223.80	246.63	314.53	408.78	249.85	297.63	314.53	542.41	240.79	260.69	314.53	430.68
Task-20-veh-05-02	287.32	287.32	484.15	474.52	287.32	326.82	484.15	710.82	287.32	291.27	484.15	607.90
Task-20-veh-05-03	287.35	291.75	360.05	340.80	296.73	315.00	360.05	498.41	294.15	298.16	360.05	420.60
Task-20-veh-05-04	222.53	218.52	472.11	371.85	245.97	263.61	472.11	561.50	235.03	246.10	472.11	462.70
Task-20-veh-05-05	726.96	736.90	777.85	832.34	742.09	761.83	777.85	1025.26	732.39	752.38	777.85	948.05
Task-50-veh-10-01	291.15	757.82	942.59	2123.97	409.85	528.75	942.59	2490.30	352.85	639.28	942.59	2265.06
Task-50-veh-10-02	592.49	879.18	1237.80	2315.96	694.02	701.22	1237.80	2690.93	633.76	773.51	1237.80	2510.30
Task-50-veh-10-03	265.47	395.08	905.72	1638.64	291.30	525.15	905.72	1916.40	279.83	465.84	905.72	1759.69
Task-50-veh-10-04	438.01	500.89	822.43	1953.96	323.74	682.12	822.43	2060.01	385.65	597.94	822.43	2014.37
Task-50-veh-10-05	384.31	593.64	937.86	2013.25	535.93	822.60	937.86	2253.15	476.28	686.99	937.86	2150.01

V. CONCLUSION

This paper introduces a heuristic MCTS-based task allocation approach for UAV swarms in reconnaissance and rescue missions. The problem's objectives and constraints are formulated using mathematical expressions. To enhance computational efficiency and solution quality, key components such as the state and action design, rollout policy, termination criteria, and rollout number are carefully tailored. The proposed approach is then benchmarked against genetic algorithms and traditional heuristic methods, demonstrating superior performance.

Future research will focus on developing learning-based rollout policies and outcome estimation methods to further enhance solution speed and quality.

REFERENCES

- [1] J. Sadaf, H. Ali, A. v. A. Waqas, A. Rehan, S. Saadat, and G. Mohsen, "State-of-the-art and future research challenges in UAV swarms," *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 19023–19045, 2024.
- [2] M. Rinaldi, S. Wang, R. S. Geronel, and S. Primatesa, "Application of task allocation algorithms in multi-UAV intelligent transportation systems: A critical review," *Big Data and Cognitive Computing*, vol. 8, no. 12, pp. 177, 2024.
- [3] H. Huang, C. Hu, J. Zhu, M. Wu, and R. Malekian, "Stochastic task scheduling in UAV-based intelligent on-demand meal delivery system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 13040–13054, 2022.
- [4] H. Dong, N. Wu, X. Gao, and G. Feng, "Research on computing task allocation method based on multi-UAVs collaboration," In *Proceedings of IEEE International Conference on Smart Internet of Things (SmartIoT)*, 2020.
- [5] Y. Luo, W. Ding, and B. Zhang, "Optimization of task scheduling and dynamic service strategy for multi-UAV-enabled mobile-edge computing system," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 3, pp. 970–984, 2021.
- [6] X. Wu, Y. Yin, L. Xu, X. Wu, F. Meng, and R. Zhen, "Multi-UAV task allocation based on improved genetic algorithm," *IEEE Access*, vol. 9, pp. 100369–100379, 2021.
- [7] H. Liu, Q. Chen, N. Pan, Y. Sun, and Y. Yang, "Three-dimensional mountain complex terrain and heterogeneous multi-UAV cooperative combat mission planning," *IEEE Access*, vol. 8, pp. 197407–197419, 2020.
- [8] H. L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 8, pp. 912–926, 2009.
- [9] B. Peng, J. Wang, and Z. Zhang, "A deep reinforcement learning algorithm using dynamic attention model for vehicle routing problems," *Artificial Intelligence Algorithms and Applications*, pp. 636–650, 2020.
- [10] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, and *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [11] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, and *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
- [12] J. Mandziuk and C. Nejman, "UCT-based approach to capacitated vehicle routing problem," *Artificial Intelligence and Soft Computing*, pp. 679–690, 2015.
- [13] M. A. Nguyen, K. Sano, and V. T. Tran, "A Monte Carlo tree search for traveling salesman problem with drone," *Asian Transport Studies*, vol. 6, 2020.
- [14] Z. Ma and J. Chen, "Multi-UAV urban logistics task allocation method based on MCTS," *Drones*, vol. 7, 2023.